TinyLlama Medusa Optimisation Report

Report date: 10 May 2026

Abstract

This report documents a Medusa-based inference acceleration project for `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. The implementation uses a local four-head Medusa configuration with tree decoding and compares standard autoregressive generation against Medusa generation, reduced-tree Turbo modes, Triton-assisted kernels, QJL-based pruning signals, KV-cache compression, and self-distilled Medusa head training.

The main finding is that Medusa performance is governed by accepted tokens per decoding step, not by tree size alone. Reduced candidate trees can lower verifier work, but they only improve end-to-end speed when they preserve the same accepted prefix depth as the full verifier. In this project, exact verification remains the reliability boundary: approximate QJL and KV-cache compression are useful as planning or memory experiments, but they are not used as final acceptance decisions.

1. Introduction

Autoregressive language-model generation is sequential: each newly generated token depends on previous tokens. This makes single-request decoding difficult to parallelise because a model normally performs one full forward pass for each new token. Medusa addresses this by adding extra decoding heads that propose future tokens in parallel. The base model then verifies the proposed tree of candidate continuations and accepts the longest prefix that matches the verifier.

The project goal was to adapt and optimise this approach for a smaller model, TinyLlama 1.1B Chat, and to understand which optimisations were useful in a single-request, batch-size-one setting. The work focused on the following questions:

1. How does Medusa change the decoding bottleneck compared with plain autoregressive generation?
2. Can a smaller Medusa tree improve throughput without reducing accepted tokens per step?
3. Can Triton kernels, QJL ranking, or KV-cache compression reduce overhead enough to improve the final path?
4. What role does Medusa head quality play in end-to-end acceleration?

2. Background

2.1 Medusa Decoding

The Medusa paper describes an inference acceleration method that adds multiple prediction heads to a base language model. These heads propose several future tokens simultaneously. Candidate paths are arranged into a tree-attention structure, and the original model verifies the candidate

continuations in a single decoding step. If several proposed tokens are accepted, the model reduces the number of full decoding steps needed to generate the same output.

This project follows the Medusa-1 style of preserving the base model as the source of truth: the verifier decides which candidate prefix is accepted. This choice is important because it keeps the optimisation focused on acceleration without changing the final greedy decoding semantics.

2.2 TinyLlama Target Model

The local Medusa configuration targets `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. The project configuration in `TinyLlama-1.1B-Chat-v1.0-4heads/config.json` specifies:

Item	Value
Base model	`TinyLlama/TinyLlama-1.1B-Chat-v1.0`
Medusa heads	4
Medusa layers	1
Decoding strategy	tree decoding
Transformers version in local config	4.40.0

TinyLlama is a compact Llama-style model family, making it suitable for local experiments where full-size LLM serving infrastructure is not available.

3. Implementation Summary

The project keeps the original Medusa structure and adds several local benchmarking and optimisation paths.

Area	Project files	Purpose
Baseline generation	`bench_base.py`, `bench_transformers_base.py`	Measure plain Hugging Face autoregressive decoding.
Medusa generation	`bench_medusa.py`, `medusa/model/medusa_model.py`	Load the four-head Medusa model and run tree decoding.
Prompt-suite benchmarking	`run_batch_benchmark.py`, `bench_comm_turbo.py`	Compare modes across prompt categories and record speed, prefix match, cache, and acceptance statistics.
Tree definitions	`medusa/model/medusa_choices.py`	Define the full Medusa tree and reduced TinyLlama presets.
Decode utilities	`medusa/model/utils.py`	Build Medusa buffers, generate candidates, evaluate posterior acceptance, and plan pruning.
Triton kernels	`medusa/model/triton_kernels.py`	Provide fused or packed GPU kernels for QJL scoring, selection, cache operations, and LM-head argmax.
KV-cache experiments	`medusa/model/kv_cache.py`	Implement FP16 cache, packed QJL sidecars, TurboVQ cache, hybrid hot-window cache, and polar cache variants.
Head training	`train_tinyllama_medusa_heads.py`, `kaggle_mix_and_train_medusa.py`	Train only Medusa heads by self-distillation while freezing the base model.

The local TinyLlama tree presets include a full Medusa choice list (`mc_sim_7b_63`), a 24-choice fast preset, and a 32-choice balanced preset. The full list contains 63 speculative choice paths; including the root verifier position, this corresponds to a 64-node verifier tree.

4. Methodology

4.1 Baseline

The baseline uses standard Hugging Face causal language-model generation with `do_sample=False`. This gives deterministic greedy decoding and provides a reference output for prefix comparison.

4.2 Medusa Base

The Medusa base path loads the local four-head Medusa model and calls `medusa_generate` with `temperature=0.0`. At this setting, the base model remains the exact verifier. The important

measurement is not just tokens per second, but accepted tokens per Medusa step. A high acceptance depth means one verifier pass can replace several autoregressive passes.

4.3 Turbo Tree Modes

The Turbo modes test reduced or adaptive trees. The main reduced-tree preset keeps the first 24 paths from the resolved Medusa tree. The intended benefit is lower verifier work per step. The risk is that removing candidate paths reduces acceptance depth, which can increase the number of decoding steps and erase the benefit of a smaller tree.

4.4 Triton-Assisted Fast Paths

The Triton work targets avoidable overhead around planning and verification:

- Greedy verification can use argmax IDs directly instead of materialising full logits for every tree position.
- QJL path scoring can use packed bit operations.
- Pruned layouts can be cached so repeated tree-mask construction is avoided.

These changes reduce overhead, but they do not change the central Medusa condition: speedup requires accepting multiple tokens per verifier step.

4.5 QJL and Packed KV-QJL

QJL is used as an approximate ranking signal. In this codebase it appears in two forms:

- A token/LM-head QJL sidecar used to help rank Medusa candidate paths.
- A packed KV-QJL sidecar using sign-projected cached keys to prefilter paths.

Both forms are treated as pre-verification signals. They are not used as final acceptance rules. Final acceptance must remain exact to preserve output correctness under greedy decoding.

4.6 KV-Cache Compression

The KV-cache experiments include FP16 cache, polar quantisation, TurboVQ, residual QJL correction, and a hybrid cache with an exact hot window. These experiments target memory and movement cost. In practice, compressed cache modes can add overhead through dequantisation, cache copying, rewinds after failed pruning, and hot-window maintenance.

4.7 Medusa Head Training

The training path freezes the base TinyLlama model and trains only the Medusa heads. The target is the base model's greedy future tokens, computed without gradients. This is aligned with greedy Medusa acceptance: a proposed token helps only when it matches the token selected by the base verifier.

The Kaggle helper builds a training mixture with these proportions:

Dataset source	Intended share
UltraChat	70%
SlimOrca	15%
CodeAlpaca	10%
GSM8K	5%

This mixture is an implementation plan in `kaggle_mix_and_train_medusa.py`. Validation metrics for this recipe are not included in the repository, so the mixture is reported as the training recipe rather than as a completed result.

5. Evaluation Metrics

The project benchmark harnesses record the following metrics:

Metric	Meaning
Tokens per second	End-to-end generated-token throughput.
Time to first token	Latency until the first generated output appears.
Accepted tokens per step	Average number of tokens accepted per Medusa decoding step.
Verified nodes per step	Average verifier tree work per step.
Prefix match vs base	Output agreement with the baseline or Medusa base reference.
Peak allocated/reserved memory	CUDA memory footprint during generation.
Estimated FP16 and compressed KV size	Approximate cache memory comparison.

The most important pair is accepted tokens per step versus verified nodes per step. A reduced tree is only a real optimisation when it lowers verified nodes without lowering accepted depth enough to require more total decode steps.

6. Findings

6.1 Full Medusa Tree Was the Reliability Anchor

The full Medusa tree preserves the widest candidate set. This improves the chance that the verifier can accept a deeper prefix in one step. The reduced 24-choice tree lowers per-step verifier cost, but the implementation treats acceptance loss and fallback verification as the main risks for the reduced path.

The final interpretation is therefore:

- The full 64-node verifier tree is the safest main reported configuration.
- The 24-choice tree is an optimisation attempt, not a replacement unless it matches prefix quality and accepted-token depth on the benchmark suite.
- Tree-size reduction should be evaluated by end-to-end tokens per second, not by node count alone.

6.2 Triton Reduced Overhead but Did Not Replace Acceptance Quality

Triton kernels are valuable for reducing avoidable GPU overhead in selection, path scoring, cache operations, and greedy LM-head argmax. However, these kernels optimise the cost of each step. They do not solve the separate problem of whether the Medusa heads propose a deep prefix that the verifier accepts.

6.3 QJL Is Useful for Ranking, Not Final Verification

The QJL path is best treated as a cheap prefilter. It can help avoid verifying weak branches, but approximate scores are not a correctness boundary. The code therefore keeps exact posterior verification after pruning. This is the right design choice for a submission report because it avoids claiming output equivalence from an approximate sketch.

6.4 KV Compression Had a Memory Motivation but a Runtime Cost

The compressed KV-cache variants reduce the theoretical cache footprint, but the runtime path can become more expensive. Dequantisation, cache copying, fallback rewinds, and hybrid hot-window maintenance all add work. If acceptance falls at the same time, the model performs more decode steps and touches the cache more often, which can erase the memory-side benefit.

6.5 Head Quality Is the Main Long-Term Bottleneck

Medusa acceleration depends on the heads predicting tokens that the base model will accept. Weak heads lower accepted tokens per step and amplify every other overhead. For new base-model transfer experiments, systems-level optimisations are meaningful only after the Medusa heads show strong top-k and accepted-prefix performance.

7. Final Configuration Recommendation

For the final submitted project result, the reliable configuration is:

Component	Recommendation
Base model	`TinyLlama/TinyLlama-1.1B-Chat-v1.0`
Medusa heads	Local four-head Medusa setup
Generation mode	Greedy Medusa generation with exact verifier acceptance
Tree	Full Medusa tree for the main reported path
Reduced tree	Report as Turbo ablation only
QJL	Use only as pruning/ranking signal before exact verification
KV compression	Report as experimental ablation, not the final fastest reliable path
Training	Freeze base model; train Medusa heads by self-distilled greedy future tokens

This recommendation keeps the report conservative and avoids unsupported claims. The central conclusion is not that every optimisation improved throughput. The central conclusion is that Medusa acceleration succeeds when speculative depth, exact verification, and cache movement are balanced.

8. Reproducibility

The benchmark scripts require a CUDA-capable environment and the TinyLlama base model weights. No generated project-results CSV was present in the repository at finalisation time, so exact throughput values are intentionally omitted rather than reconstructed from memory.

Recommended commands for reproducing the main measurements are:

```
python bench_transformers_base.py \
  --model-dir TinyLlama/TinyLlama-1.1B-Chat-v1.0 \
  --out-csv base_transformers_benchmark.csv \
  --target-new-tokens 160 \
  --prompt-suite mixed
```

```
python bench_comm_turbo.py \
  --model-dir TinyLlama-1.1B-Chat-v1.0-4heads \
  --out-csv comm_turbo_benchmark.csv \
  --target-new-tokens 160 \
  --prompt-suite mixed \
  --choice-sweep 24,32
```

```
python run_batch_benchmark.py
```

Generated CSV files are the source of truth for exact averaged throughput, prefix-match, acceptance, and memory values. This report does not invent missing throughput numbers.

9. Conclusion

The TinyLlama Medusa optimisation project shows that tree decoding is valuable only when the speculative heads allow the verifier to accept multiple tokens per step. The most reliable path keeps the base model as the exact verifier and uses the full Medusa tree as the main configuration. Reduced trees, QJL planning, and KV-cache compression are meaningful ablations, but they must preserve accepted depth to improve real throughput.

The final research conclusion is:

> Medusa optimisation should prioritise accepted tokens per step under exact verification. Node-count reduction, approximate ranking, and cache compression are secondary optimisations that help only when they do not reduce verifier acceptance or add more overhead than they remove.

References

1. Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads.” arXiv:2401.10774, 2024. https://arxiv.org/abs/2401.10774
2. Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. “TinyLlama: An Open-Source Small Language Model.” arXiv:2401.02385, 2024. https://arxiv.org/abs/2401.02385
3. TinyLlama model card for `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0
4. Triton documentation. https://triton-lang.org/main/index.html